

How to Use This Coding Guide

The SP/DSE coding round has 3 problems (Easy/Medium/Hard) over 3 hours, with a strong emphasis on Dynamic Programming and Greedy techniques; Specialist Programmer problems trend to competitive-programming difficulty.

Problems are organised by pattern. Each Python solution has been executed and verified against its sample output.

Authenticity labels appear next to each problem's difficulty.

Patterns covered: Arrays (4), Strings (3), Hashing (2), Sliding Window (2), Two Pointers (2), Binary Search (2), Stack (2), Greedy (2), Dynamic Programming (6), Recursion & Backtracking (2), Math & Number Theory (3)

Pattern: Arrays

Problem 1: Second Largest Element

Difficulty: Easy | Pattern: Array Traversal | Authenticity: Community Reported

Problem Statement

Find the second largest element in an array. Print -1 if it does not exist.

Input Format

Line 1: N. Line 2: N integers.

Output Format

The second largest element or -1.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
6
12 35 1 10 34 1
```

Sample Output

```
34
```

Python Solution

```
def second_largest(arr):
    largest = second = float('-inf')
    for x in arr:
        if x > largest:
            second, largest = largest, x
        elif largest > x > second:
            second = x
    return second if second != float('-inf') else -1

print(second_largest([12, 35, 1, 10, 34, 1]))
```

Step-by-Step Explanation

Track largest and second largest in one pass; a new maximum pushes the old maximum into second place.

Dry Run

35->largest;12->second;34 sits between 12 and 35->second=34. Result 34.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 2: Maximum Subarray Sum (Kadane)

Difficulty: *Easy-Medium* | Pattern: *Array / DP* | Authenticity: *Frequently Repeated*

Problem Statement

Find the maximum sum of any contiguous non-empty subarray.

Input Format

Line 1: N. Line 2: N integers.

Output Format

Maximum contiguous subarray sum.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Sample Output

```
6
```

Python Solution

```
def max_subarray(arr):
    best = cur = arr[0]
    for x in arr[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

print(max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
```

Step-by-Step Explanation

At each element extend the running sum or restart; track the best. Subarray [4,-1,2,1] gives 6.

Dry Run

cur peaks at 6 over [4, -1, 2, 1]; best=6.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 3: Move All Zeroes to End

Difficulty: *Easy* | Pattern: *In-place Two Pointers* | Authenticity: *Community Reported*

Problem Statement

Move all zeroes to the end while keeping the order of non-zero elements.

Input Format

Line 1: N. Line 2: N integers.

Output Format

Modified array, space-separated.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
7
0 1 0 3 12 0 5
```

Sample Output

```
1 3 12 5 0 0 0
```

Python Solution

```
def move_zeroes(arr):
    pos = 0
    for x in arr:
        if x != 0:
            arr[pos] = x; pos += 1
    while pos < len(arr):
        arr[pos] = 0; pos += 1
    return arr

print(' '.join(map(str, move_zeroes([0, 1, 0, 3, 12, 0, 5]))))
```

Step-by-Step Explanation

A write pointer copies non-zero values forward; remaining slots are filled with zeroes.

Dry Run

Non-zeros 1,3,12,5 written first; then zeroes fill positions 4-6.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 4: Rotate Array by K

Difficulty: *Easy* | Pattern: *Array / Slicing* | Authenticity: *Community Reported*

Problem Statement

Rotate an array to the right by k positions.

Input Format

Line 1: N and K. Line 2: N integers.

Output Format

Rotated array, space-separated.

Constraints

$1 \leq N \leq 10^5, 0 \leq K$

Sample Input

```
7 3
1 2 3 4 5 6 7
```

Sample Output

```
5 6 7 1 2 3 4
```

Python Solution

```
def rotate(arr, k):
    n = len(arr)
    k %= n
    return arr[-k:] + arr[:-k]

print(' '.join(map(str, rotate([1, 2, 3, 4, 5, 6, 7], 3))))
```

Step-by-Step Explanation

Rotating right by k means the last k elements move to the front. Taking k modulo n handles k larger than n .

Dry Run

Last 3 [5,6,7] move front; result 5 6 7 1 2 3 4.

Time Complexity: **$O(N)$** | Space Complexity: **$O(N)$**

Pattern: Strings

Problem 1: Reverse Words in a Sentence

Difficulty: **Easy** | Pattern: **String Manipulation** | Authenticity: **Community Reported**

Problem Statement

Reverse the order of words in a sentence.

Input Format

A single line sentence.

Output Format

Sentence with word order reversed.

Constraints

$1 \leq \text{length} \leq 10^4$

Sample Input

```
data structures algorithms
```

Sample Output

```
algorithms structures data
```

Python Solution

```
def reverse_words(s):  
    return ' '.join(reversed(s.split()))  
  
print(reverse_words('data structures algorithms'))
```

Step-by-Step Explanation

split() tokenises on whitespace, reversed() flips order, join() rebuilds.

Dry Run

```
['data', 'structures', 'algorithms'] -> reversed -> joined.
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 2: Valid Anagram

Difficulty: *Easy* | Pattern: *Frequency Counting* | Authenticity: *Community Reported*

Problem Statement

Check whether two strings are anagrams of each other.

Input Format

Line 1: first string. Line 2: second string.

Output Format

True or False.

Constraints

1 <= length <= 10⁴

Sample Input

```
listen  
silent
```

Sample Output

```
True
```

Python Solution

```
def is_anagram(a, b):  
    if len(a) != len(b):  
        return False  
    counts = {}  
    for ch in a:  
        counts[ch] = counts.get(ch, 0) + 1  
    for ch in b:  
        if counts.get(ch, 0) == 0:  
            return False  
        counts[ch] -= 1  
    return True  
  
print(is_anagram('listen', 'silent'))
```

Step-by-Step Explanation

Count characters of the first string, decrement for the second; equal counts mean anagram.

Dry Run

All letters of 'silent' cancel counts from 'listen' -> True.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 3: First Non-Repeating Character

Difficulty: *Easy* | Pattern: *Hashing on Strings* | Authenticity: *Community Reported*

Problem Statement

Return the index of the first character that appears exactly once. Print -1 if none.

Input Format

A single line string.

Output Format

Index of first unique character or -1.

Constraints

$1 \leq \text{length} \leq 10^5$

Sample Input

```
leetcode
```

Sample Output

```
0
```

Python Solution

```
from collections import Counter
def first_unique(s):
    freq = Counter(s)
    for i, ch in enumerate(s):
        if freq[ch] == 1:
            return i
    return -1

print(first_unique('leetcode'))
```

Step-by-Step Explanation

Count frequencies, then scan left to right for the first character with count 1.

Dry Run

'l' occurs once and is first -> index 0.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Pattern: Hashing

Problem 1: Two Sum

Difficulty: *Easy* | Pattern: *Hash Map* | Authenticity: *Frequently Repeated*

Problem Statement

Return indices of the two numbers that add up to the target.

Input Format

Line 1: array. Line 2: target.

Output Format

Two indices, space-separated.

Constraints

$2 \leq N \leq 10^5$

Sample Input

```
2 7 11 15
9
```

Sample Output

```
0 1
```

Python Solution

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i

i, j = two_sum([2, 7, 11, 15], 9)
print(i, j)
```

Step-by-Step Explanation

Store each value's index; when the complement is found, return the pair.

Dry Run

2 seen; at 7 complement 2 found -> (0,1).

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 2: Count Pairs with Given Sum

Difficulty: **Easy-Medium** | Pattern: **Hash Map** | Authenticity: **Community Reported**

Problem Statement

Count the number of pairs (i, j), $i < j$, whose values sum to the target.

Input Format

Line 1: array. Line 2: target.

Output Format

Number of such pairs.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
1 5 7 -1 5
6
```

Sample Output

```
3
```

Python Solution

```
def count_pairs(arr, target):
    seen = {}
    count = 0
    for x in arr:
        count += seen.get(target - x, 0)
        seen[x] = seen.get(x, 0) + 1
    return count

print(count_pairs([1, 5, 7, -1, 5], 6))
```

Step-by-Step Explanation

For each element add the number of earlier elements equal to its complement, then record it. Pairs: (1,5),(7,-1),(1,5).

Dry Run

counts accumulate to 3.

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Pattern: Sliding Window

Problem 1: Longest Substring Without Repeating Characters

Difficulty: **Medium** | Pattern: **Sliding Window + Hash Map** | Authenticity: **Community Reported**

Problem Statement

Find the length of the longest substring with all distinct characters.

Input Format

A single line string.

Output Format

Length of the longest such substring.

Constraints

$0 \leq \text{length} \leq 10^5$

Sample Input

```
abcabcbb
```

Sample Output

```
3
```


Python Solution

```
def longest_unique(s):
    last = {}
    start = best = 0
    for i, ch in enumerate(s):
        if ch in last and last[ch] >= start:
            start = last[ch] + 1
        last[ch] = i
        best = max(best, i - start + 1)
    return best

print(longest_unique('abcabcbb'))
```

Step-by-Step Explanation

Maintain a window; on a repeat, jump start past the previous occurrence.

Dry Run

'abc' length 3 is the longest.

Time Complexity: **O(N)** | Space Complexity: **O(min(N, alphabet))**

Problem 2: Maximum Sum Subarray of Size K

Difficulty: **Easy-Medium** | Pattern: **Sliding Window** | Authenticity: **Community Reported**

Problem Statement

Find the maximum sum of any contiguous subarray of exactly size k.

Input Format

Line 1: N and K. Line 2: N integers.

Output Format

Maximum window sum.

Constraints

$1 \leq K \leq N \leq 10^5$

Sample Input

```
6 3
2 1 5 1 3 2
```

Sample Output

```
9
```

Python Solution

```
def max_sum_k(arr, k):
    window = sum(arr[:k])
    best = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i - k]
        best = max(best, window)
    return best

print(max_sum_k([2, 1, 5, 1, 3, 2], 3))
```

Step-by-Step Explanation

Slide a fixed window, adding the new element and removing the old one.

Dry Run

Window [5,1,3] gives 9.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Pattern: Two Pointers

Problem 1: Trapping Rain Water

Difficulty: **Hard** | Pattern: **Two Pointers** | Authenticity: **Community Reported**

Problem Statement

Given an elevation map, compute how much rain water can be trapped.

Input Format

Line 1: N. Line 2: N heights.

Output Format

Total trapped water units.

Constraints

$0 \leq N \leq 10^5$

Sample Input

```
12
0 1 0 2 1 0 1 3 2 1 2 1
```

Sample Output

```
6
```

Python Solution

```
def trap(height):
    l, r = 0, len(height) - 1
    lmax = rmax = water = 0
    while l < r:
        if height[l] < height[r]:
            lmax = max(lmax, height[l]); water += lmax - height[l]; l += 1
        else:
            rmax = max(rmax, height[r]); water += rmax - height[r]; r -= 1
    return water

print(trap([0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]))
```

Step-by-Step Explanation

Water over a bar is bounded by the smaller of the tallest bars on each side; advance the shorter side.

Dry Run

Sum of per-index contributions = 6.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 2: Pair with Target Sum (Sorted)

Difficulty: **Easy** | Pattern: **Two Pointers** | Authenticity: **Community Reported**

Problem Statement

In a sorted array, decide whether two elements sum to the target.

Input Format

Line 1: sorted array. Line 2: target.

Output Format

True or False.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
1 2 4 7 11 15
15
```

Sample Output

```
True
```

Python Solution

```
def has_pair(arr, target):
    i, j = 0, len(arr) - 1
    while i < j:
        s = arr[i] + arr[j]
        if s == target:
            return True
        if s < target:
            i += 1
        else:
            j -= 1
    return False

print(has_pair([1, 2, 4, 7, 11, 15], 15))
```

Step-by-Step Explanation

Move inward: increase the left pointer to grow the sum, decrease the right to shrink it. $4 + 11 = 15$.

Dry Run

i, j converge to 4 and 11 $\rightarrow$ True.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Pattern: Binary Search

Problem 1: Binary Search

Difficulty: *Easy* | Pattern: *Binary Search* | Authenticity: *Community Reported*

Problem Statement

Return the index of the target in a sorted array, or -1.

Input Format

Line 1: sorted array. Line 2: target.

Output Format

Index or -1.

Constraints

$1 \leq N \leq 10^6$

Sample Input

```
1 3 5 7 9 11
7
```

Sample Output

```
3
```

Python Solution

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9, 11], 7))
```

Step-by-Step Explanation

Halve the search range each step by comparing with the middle element.

Dry Run

mid lands on index 3 (value 7).

Time Complexity: **$O(\log N)$** | Space Complexity: **$O(1)$**

Problem 2: Search in Rotated Sorted Array

Difficulty: *Medium* | Pattern: *Modified Binary Search* | Authenticity: *Community Reported*

Problem Statement

Search a target in a sorted array that has been rotated at an unknown pivot; return the index or -1.

Input Format

Line 1: array. Line 2: target.

Output Format

Index or -1.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
4 5 6 7 0 1 2
0
```

Sample Output

```
4
```

Python Solution

```
def search_rotated(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        if arr[lo] <= arr[mid]:
            if arr[lo] <= target < arr[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:
            if arr[mid] < target <= arr[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0))
```

Step-by-Step Explanation

One half is always sorted; check whether the target lies within it and discard the other half.

Dry Run

Target 0 is found at index 4.

Time Complexity: **$O(\log N)$** | Space Complexity: **$O(1)$**

Pattern: Stack

Problem 1: Valid Parentheses

Difficulty: *Easy* | Pattern: *Stack* | Authenticity: *Community Reported*

Problem Statement

Determine whether a bracket string is validly matched and nested.

Input Format

A single line of brackets.

Output Format

True or False.

Constraints

$1 \leq \text{length} \leq 10^4$

Sample Input

```
()[]{}
```

Sample Output

```
True
```

Python Solution

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif not stack or stack.pop() != pairs[ch]:
            return False
    return not stack

print(is_valid('()[]{}'))
```

Step-by-Step Explanation

Push openers; each closer must match the top of the stack. Empty stack at the end means valid.

Dry Run

Each pair matches and stack ends empty -> True.

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 2: Next Greater Element

Difficulty: **Medium** | Pattern: **Monotonic Stack** | Authenticity: **Community Reported**

Problem Statement

For each element, find the next element to its right that is greater; use -1 if none.

Input Format

Line 1: N. Line 2: N integers.

Output Format

The result array, space-separated.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
4
4 5 2 25
```

Sample Output

```
5 25 25 -1
```

Python Solution

```
def next_greater(arr):
    res = [-1] * len(arr)
    stack = []
    for i, x in enumerate(arr):
        while stack and arr[stack[-1]] < x:
            res[stack.pop()] = x
        stack.append(i)
    return res

print(' '.join(map(str, next_greater([4, 5, 2, 25]))))
```

Step-by-Step Explanation

A decreasing stack of indices; a larger value resolves all smaller pending elements.

Dry Run

4->5, 5->25, 2->25, 25->-1.

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Pattern: Greedy

Problem 1: Jump Game (Reachability)

Difficulty: **Medium** | Pattern: **Greedy** | Authenticity: **Community Reported**

Problem Statement

Each value is the maximum jump length from that index. Decide whether you can reach the last index starting from index 0.

Input Format

Line 1: N. Line 2: N integers.

Output Format

True or False.

Constraints

$1 \leq N \leq 10^4$

Sample Input

```
5
2 3 1 1 4
```

Sample Output

```
True
```

Python Solution

```
def can_jump(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + x)
    return True

print(can_jump([2, 3, 1, 1, 4]))
```

Step-by-Step Explanation

Greedily track the furthest reachable index; if an index is beyond it, the end is unreachable.

Dry Run

reach grows 0->2->4; last index reachable -> True.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 2: Best Time to Buy and Sell Stock

Difficulty: **Easy** | Pattern: **Greedy / One Pass** | Authenticity: **Community Reported**

Problem Statement

With one buy and one later sell, find the maximum profit (0 if none).

Input Format

Line 1: N. Line 2: N prices.

Output Format

Maximum profit.

Constraints

$1 \leq N \leq 10^5$

Sample Input

```
6
7 1 5 3 6 4
```

Sample Output

```
5
```

Python Solution

```
def max_profit(prices):
    min_p = float('inf')
    best = 0
    for p in prices:
        min_p = min(min_p, p)
        best = max(best, p - min_p)
    return best

print(max_profit([7, 1, 5, 3, 6, 4]))
```

Step-by-Step Explanation

Track the minimum price so far; best profit is current price minus that minimum. Buy 1, sell 6.

Dry Run

min hits 1; best profit 6-1=5.

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Pattern: Dynamic Programming

Problem 1: Coin Change (Minimum Coins)

Difficulty: **Medium** | Pattern: **DP (Unbounded)** | Authenticity: **Frequently Repeated**

Problem Statement

Minimum number of coins to make an amount; coins reusable; -1 if impossible.

Input Format

Line 1: number of coins and amount. Line 2: denominations.

Output Format

Minimum coins or -1.

Constraints

1 <= amount <= 10⁴

Sample Input

```
3 11
1 2 5
```

Sample Output

```
3
```

Python Solution

```
def coin_change(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1

print(coin_change([1, 2, 5], 11))
```

Step-by-Step Explanation

dp[a] = fewest coins for amount a; try every coin for every amount. 11 = 5+5+1.

Dry Run

dp[11] resolves to 3.

Time Complexity: **O(amount*coins)** | Space Complexity: **O(amount)**

Problem 2: Longest Increasing Subsequence

Difficulty: **Medium** | Pattern: **DP** | Authenticity: **Frequently Repeated**

Problem Statement

Length of the longest strictly increasing subsequence.

Input Format

Line 1: N. Line 2: N integers.

Output Format

LIS length.

Constraints

$1 \leq N \leq 2500$

Sample Input

```
8
10 9 2 5 3 7 101 18
```

Sample Output

```
4
```

Python Solution

```
def length_of_lis(arr):
    n = len(arr)
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

print(length_of_lis([10, 9, 2, 5, 3, 7, 101, 18]))
```

Step-by-Step Explanation

$dp[i]$ is the LIS ending at i ; extend from any smaller earlier element.

Dry Run

[2,3,7,101] gives length 4.

Time Complexity: **$O(N^2)$** | Space Complexity: **$O(N)$**

Problem 3: 0/1 Knapsack

Difficulty: **Medium-Hard** | Pattern: **DP** | Authenticity: **Frequently Repeated**

Problem Statement

Maximise value within a weight capacity, each item used at most once.

Input Format

Line 1: N and W. Line 2: weights. Line 3: values.

Output Format

Maximum achievable value.

Constraints

$1 \leq N \leq 100$, $1 \leq W \leq 1000$

Sample Input

```
3 50
10 20 30
60 100 120
```

Sample Output

```
220
```

Python Solution

```
def knapsack(weights, values, cap):
    dp = [0] * (cap + 1)
    for i in range(len(weights)):
        for w in range(cap, weights[i] - 1, -1):
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
    return dp[cap]

print(knapsack([10, 20, 30], [60, 100, 120], 50))
```

Step-by-Step Explanation

Iterate capacity high-to-low so each item is counted once. Items 2+3 (20+30) give 220.

Dry Run

dp[50] becomes 220.

Time Complexity: $O(N*W)$ | Space Complexity: $O(W)$

Problem 4: Edit Distance (Levenshtein)

Difficulty: **Hard** | Pattern: **DP (2D)** | Authenticity: **Community Reported**

Problem Statement

Minimum insert/delete/replace operations to convert word1 into word2.

Input Format

Line 1: word1. Line 2: word2.

Output Format

Minimum edit distance.

Constraints

$0 \leq \text{length} \leq 500$

Sample Input

```
horse
ros
```

Sample Output

```
3
```

Python Solution

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[m][n]

print(edit_distance('horse', 'ros'))
```

Step-by-Step Explanation

$dp[i][j]$ is the distance between prefixes; matching chars cost nothing, else 1 plus the best neighbour.

Dry Run

horse -> ros needs 3 operations.

Time Complexity: **$O(m \cdot n)$** | Space Complexity: **$O(m \cdot n)$**

Problem 5: Subset Sum

Difficulty: **Medium** | Pattern: **DP (0/1)** | Authenticity: **Verified Previous-Year**

Problem Statement

Given a set of non-negative integers, decide if any subset sums to a given target. (Reported on Infosys SP coding sets.)

Input Format

Line 1: N and target. Line 2: N integers.

Output Format

True or False.

Constraints

$1 \leq N \leq 100$, $0 \leq \text{target} \leq 10^4$

Sample Input

```
6 9
3 34 4 12 5 2
```

Sample Output

True

Python Solution

```
def subset_sum(nums, target):
    dp = [False] * (target + 1)
    dp[0] = True
    for x in nums:
```

```

        for s in range(target, x - 1, -1):
            if dp[s - x]:
                dp[s] = True
        return dp[target]

print(subset_sum([3, 34, 4, 12, 5, 2], 9))

```

Step-by-Step Explanation

dp[s] is True if some subset sums to s; process each number once (iterate sums downward). Subset {4,5} sums to 9.

Dry Run

dp[9] becomes True via 4+5.

Time Complexity: **O(N*target)** | Space Complexity: **O(target)**

Problem 6: Split Array into K Segments (Minimum Cost)

Difficulty: **Hard** | Pattern: **Partition DP** | Authenticity: **Verified Previous-Year**

Problem Statement

Split an array into exactly k consecutive segments so the total cost is minimum, where a segment's cost is (max - min) of its elements. (Based on a reported Infosys SP partitioning problem; cost defined here for a self-contained version.)

Input Format

Line 1: N and K. Line 2: N integers.

Output Format

Minimum total cost.

Constraints

1 <= K <= N <= 500

Sample Input

```

5 2
1 3 6 7 9

```

Sample Output

```

5

```

Python Solution

```

def min_split_cost(arr, k):
    n = len(arr)
    INF = float('inf')
    dp = [[INF] * (k + 1) for _ in range(n + 1)]
    dp[0][0] = 0
    for i in range(1, n + 1):
        for j in range(1, k + 1):
            seg_min, seg_max = INF, -INF
            for t in range(i, 0, -1):
                seg_min = min(seg_min, arr[t - 1])
                seg_max = max(seg_max, arr[t - 1])
                if dp[t - 1][j - 1] != INF:
                    cost = dp[t - 1][j - 1] + (seg_max - seg_min)

```

```

        dp[i][j] = min(dp[i][j], cost)
    return dp[n][k]

print(min_split_cost([1, 3, 6, 7, 9], 2))

```

Step-by-Step Explanation

$dp[i][j]$ = min cost to split the first i elements into j segments. For each split point we try the last segment $arr[t..i-1]$. Best split is $[1,3]$ and $[6,7,9]$ with cost $2 + 3 = 5$.

Dry Run

$dp[5][2]$ resolves to 5.

Time Complexity: $O(N^2 * K)$ | Space Complexity: $O(N * K)$

Pattern: Recursion & Backtracking

Problem 1: Generate All Subsets (Power Set)

Difficulty: **Medium** | Pattern: **Backtracking** | Authenticity: **Community Reported**

Problem Statement

Generate the power set of a list; here we report its size.

Input Format

Line 1: N . Line 2: N integers.

Output Format

Number of subsets.

Constraints

$0 \leq N \leq 20$

Sample Input

```

3
1 2 3

```

Sample Output

```

8

```

Python Solution

```

def subsets(nums):
    res = [[]]
    for x in nums:
        res += [cur + [x] for cur in res]
    return res

print(len(subsets([1, 2, 3])))

```

Step-by-Step Explanation

Each new element either joins or skips every existing subset, doubling the count. $2^3 = 8$.

Dry Run

Sizes double: 1 -> 2 -> 4 -> 8.

Time Complexity: $O(N * 2^N)$ | Space Complexity: $O(N * 2^N)$

Problem 2: Generate Balanced Parentheses

Difficulty: **Medium** | Pattern: **Backtracking** | Authenticity: **Community Reported**

Problem Statement

Count the distinct well-formed parenthesis strings using n pairs.

Input Format

A single integer n .

Output Format

The count of valid combinations.

Constraints

$0 \leq n \leq 12$

Sample Input

3

Sample Output

5

Python Solution

```
def gen_parentheses(n):
    res = []
    def bt(cur, opened, closed):
        if len(cur) == 2 * n:
            res.append(cur); return
        if opened < n:
            bt(cur + '(', opened + 1, closed)
        if closed < opened:
            bt(cur + ')', opened, closed + 1)
    bt('', 0, 0)
    return len(res)
```

```
print(gen_parentheses(3))
```

Step-by-Step Explanation

Add '(' while available and ')' only when it can close an open bracket. For $n=3$ there are 5 valid strings (the 3rd Catalan number).

Dry Run

Valid: ((())), (()()), ()()(), ()(()), ()()() -> 5.

Time Complexity: $O(4^n / \sqrt{n})$ | Space Complexity: $O(n)$

Pattern: Math & Number Theory

Problem 1: Count Primes (Sieve of Eratosthenes)

Difficulty: **Medium** | Pattern: **Sieve** | Authenticity: **Community Reported**

Problem Statement

Count the number of prime numbers strictly less than n .

Input Format

A single integer n .

Output Format

Count of primes below n .

Constraints

$0 \leq n \leq 5 \cdot 10^6$

Sample Input

10

Sample Output

4

Python Solution

```
def count_primes(n):
    if n < 2:
        return 0
    sieve = [True] * n
    sieve[0] = sieve[1] = False
    for i in range(2, int(n ** 0.5) + 1):
        if sieve[i]:
            for j in range(i * i, n, i):
                sieve[j] = False
    return sum(sieve)

print(count_primes(10))
```

Step-by-Step Explanation

Mark multiples of each prime as composite; the survivors are primes. Below 10: 2,3,5,7.

Dry Run

4 primes remain.

Time Complexity: **$O(n \log \log n)$** | Space Complexity: **$O(n)$**

Problem 2: Prime Pairs Summing to N (Goldbach)

Difficulty: **Medium** | Pattern: **Number Theory** | Authenticity: **Community Reported**

Problem Statement

Count unordered pairs of primes (p, q) with $p + q = n$ and $p \leq q$. (Relates to reported 'prime number pairs' problems.)

Input Format

A single integer n .

Output Format

Number of prime pairs.

Constraints

$1 \leq n \leq 10^5$

Sample Input

10

Sample Output

2

Python Solution

```
def prime_pairs(n):
    def is_prime(x):
        if x < 2:
            return False
        i = 2
        while i * i <= x:
            if x % i == 0:
                return False
            i += 1
        return True
    count = 0
    for p in range(2, n // 2 + 1):
        if is_prime(p) and is_prime(n - p):
            count += 1
    return count

print(prime_pairs(10))
```

Step-by-Step Explanation

For each p up to $n/2$, check if both p and $n-p$ are prime. For 10: (3,7) and (5,5).

Dry Run

2 valid prime pairs.

Time Complexity: $O(n * \sqrt{n})$ | Space Complexity: $O(1)$

Problem 3: LCM of Two Numbers

Difficulty: **Easy** | Pattern: **Number Theory (GCD)** | Authenticity: **Community Reported**

Problem Statement

Compute the least common multiple of two positive integers.

Input Format

Two integers a and b .

Output Format

The LCM.

Constraints

$1 \leq a, b \leq 10^9$

Sample Input

4 6

Sample Output

12

Python Solution

```
import math
def lcm(a, b):
    return a * b // math.gcd(a, b)

print(lcm(4, 6))
```

Step-by-Step Explanation

$LCM(a,b) = a*b / GCD(a,b)$. $GCD(4,6)=2$, so $LCM = 24/2 = 12$.

Dry Run

gcd=2 -> lcm=12.

Time Complexity: **$O(\log(\min(a,b)))$** | Space Complexity: **$O(1)$**

Infosys

Coding Problems (Pattern-wise)

Applied Role(s):

Digital Specialist Engineer (Trainee) | Specialist Programmer L1 / L2 / L3 (Trainee)

Online Assessment Preparation Material

Version 1.1 • Last Updated: July 2026

Every question carries an authenticity label. See the legend on the next page.

Table of Contents

How to Use This Coding Guide	1
Pattern: Arrays	1
Pattern: Strings	4
Pattern: Hashing	6
Pattern: Sliding Window	8
Pattern: Two Pointers	10
Pattern: Binary Search	11
Pattern: Stack	13
Pattern: Greedy	15
Pattern: Dynamic Programming	17
Pattern: Recursion & Backtracking	22
Pattern: Math & Number Theory	23
Authenticity Legend & How to Read This Document	28

Authenticity Legend & How to Read This Document

Every question and problem in this document is labelled with an **authenticity classification** so you know how closely it maps to what candidates have actually reported. Labels are assigned conservatively based on research; authored practice questions are never presented as real exam questions.

Label	Meaning
Verified Previous-Year	The specific problem/pattern is documented from actual reported assessment experiences.
Frequently Repeated	The topic or pattern type is documented as recurring across multiple hiring cycles.
Community Reported	A widely shared, role-relevant pattern reported across placement communities and forums.
Research-Backed Company Style	Authored to match the researched difficulty, style and pattern - practice, not a real past question.

The correct option in every MCQ is highlighted in green and also restated below the question. Study the pattern, not just the answer.